



编译原理

张 南

nanzhang@xidian.edu.cn

西安电子科技大学



一、编译原理课程定位

编译原理课程：

- (1) 计算机科学与技术、软件工程专业，以及计算机系统安全有关专业的核心基础课程；
- (2) 培养解决复杂软件问题能力的重要课程之一；
- (3) 理解程序设计语言运行机理的重要课程。

编译系统：

- (1) 将源语言程序转换为目标语言程序；
- (2) 桥接了软、硬件平台；
- (3) 是建立硬件系统软件生态的核心，是基础软件领域和程序安全领域面临的“卡脖子”问题之一。



课程目标：

1. 了解程序设计语言的基本要素和运行机理；
2. 掌握编译的基本概念、编译器实现的原理和基本方法；
3. 具备独立编写一个小型编译系统的能力；
4. 了解计算机科学的基本理论。

二、学习方法

2.1 本课程的特点

- 理论与实践并重
- 理论学习要严谨、方法掌握要灵活
- 提高自学能力

2.2 理论与技术的关系

- 适应飞速变化的技术的根本是注重基础理论学习
- 理论的演变是缓慢的、理论基础是相通的
- 相同的原理可以应用于不同的技术



2.3 成绩

平时作业 10%

上机实践 20%

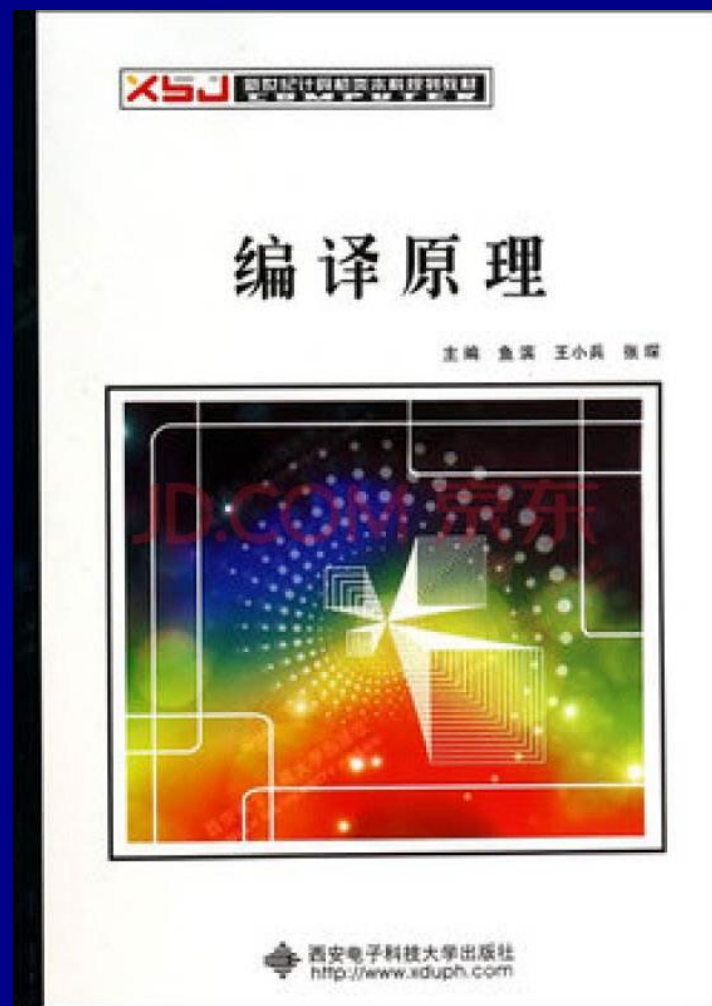
试卷成绩 70%

2.4 教材

- 编译原理

主编：鱼滨、王小兵、张琛

西安电子科技大学出版社





2.5 参考书目

中文:

- 清华大学出版社, 吕映芝等, “编译原理”
- 国防工业出版社, 陈火旺等, “程序设计语言编译原理”
- 西北工业大学出版社, 蒋立源等, “编译原理”

英文:

- 人民邮电出版社, Aho等, “编译原理技术与工具” (影印版) ——龙书
- 高等教育出版社, Andrew W. Appel, “现代编译程序实现—Java语言” (影印版) ——虎书
- 机械工业出版社, Steven S. Muchnick, “高级编译器设计与实现” (影印版) ——鲸书
- 清华大学出版社, Hopcroft等, “自动机理论、语言和计算机导论” (影印版)



第一章 引言

1.1 从面向机器的语言到面向人类的语言

面向机器的语言：机器指令、汇编语言

面向人类的语言：通用程序设计语言、非过程式语言，等等

<1> 计算机语言举例

例1： 通用程序设计语言与汇编语言（包括机器指令）

Pascal语句： $x := a+b;$

C++语句： $x = a+b;$

汇编指令： 十六进制代码

汇编指令

A10002

MOV AX, [A]

8B1E0202

MOV BX, [B]

01D8

ADD AX, BX

A30402

MOV [X], AX



例2 SQL 学生:

学号	姓名	性别
001	张梧	男
002	李煦	男
003	王沁	女
004	刘荔	女

选课:

学号	课程代码	课程名	成绩
001	0104	离散数学	80
001	0205	数据结构	90
003	0104	离散数学	85
003	0205	数据结构	95

给出003号学生所选课程与成绩:

Select 学生. 学号, 姓名, 课程名, 成绩
from 学生, 选课
where 学生. 学号='003' and 学生. 学号=选课. 学号

学号	姓名	课程名	成绩
003	王沁	离散数学	85
003	王沁	数据结构	95



例3: LEX的正规式: $\text{char}(\text{char}|\text{digit})^*$

Yacc的产生式: $E : E '+' E \mid E '*' E \mid \text{id}$

<2> 按范型划分的程序设计语言

清华大学出版社, Terrence W. Pratt等, "PROGRAMMING LANGUAGES Design and Implementation" (影印版)

1. Simple Procedural Languages: FORTRAN C
2. Block-Structured Procedural Languages: Pascal
3. Object-Based Languages: Ada C++ Smalltalk
4. Functional Languages: LISP ML Haskell
5. Logic Programming Languages: Prolog



<3> 其他面向特定应用领域的语言

1. 互连网应用：HTML、XML
2. 计算机辅助设计：MATLAB
3. 集成电路设计：VHDL、Verilog
4. 虚拟现实与人机交互：VRML

.....

问题：

如何将形形色色的语言翻译成可以在计算机上运行的0、1串？

答：面向人类的语言的产生导致了编译技术的产生。

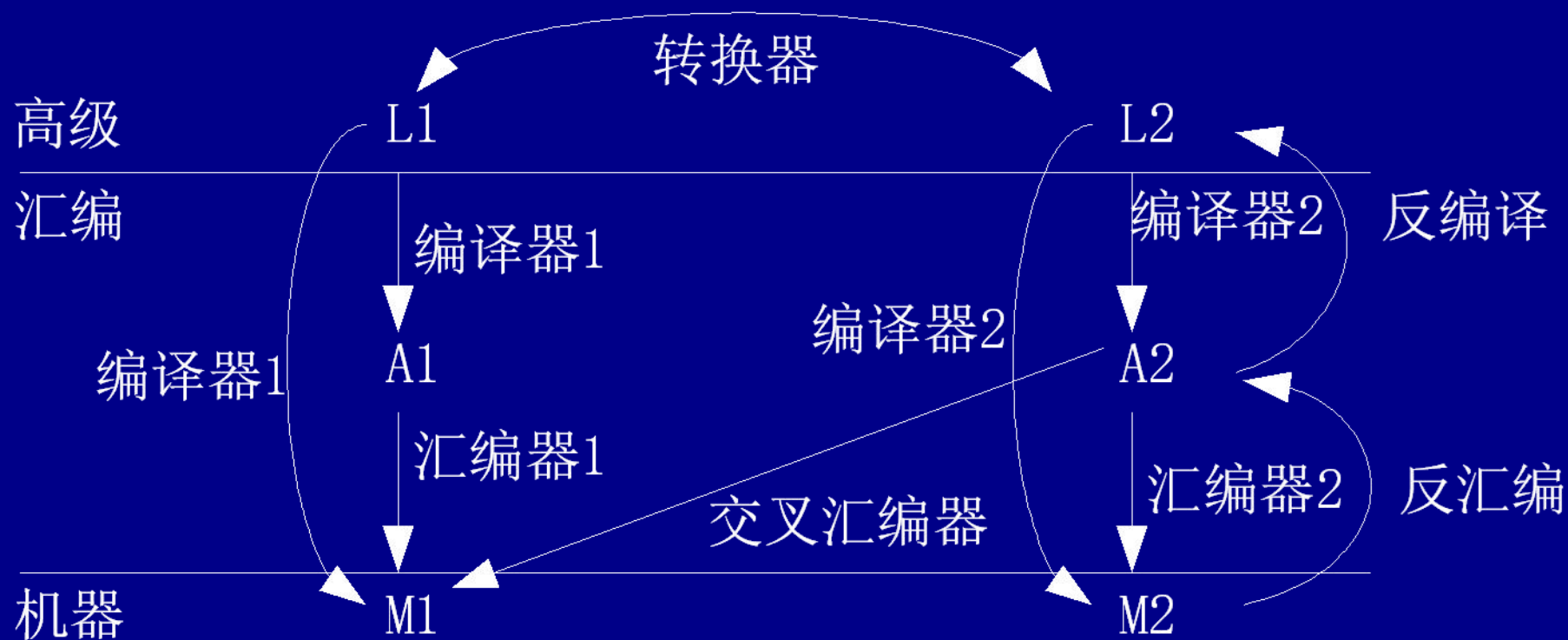
编译：将面向人类的高级语言（符合人类思维方式的、用文字描述的高级语言源程序）翻译成面向机器的低级语言（计算机能够理解和执行的低级语言目标程序）的过程。

问题：

由于语言的层次不同（高级语言、汇编语言、机器语言），语言之间的翻译还有哪些其他形式呢？



1.2 语言之间的翻译



习惯称法:

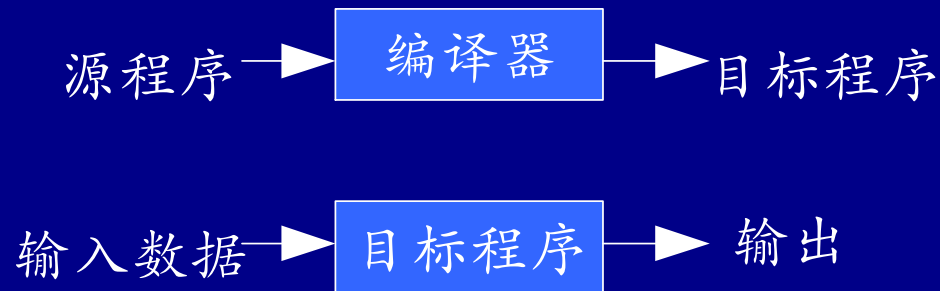
- <1> 汇编语言—机器指令: 汇编 (或交叉汇编)
- <2> 程序设计语言—汇编语言或机器指令: 编译 (或解释)
- <3> 高级语言之间: 转换 (或预编译)
- <4> 逆向: 反汇编、反编译



1.3 编译器与解释器

语言翻译的两种基本形态

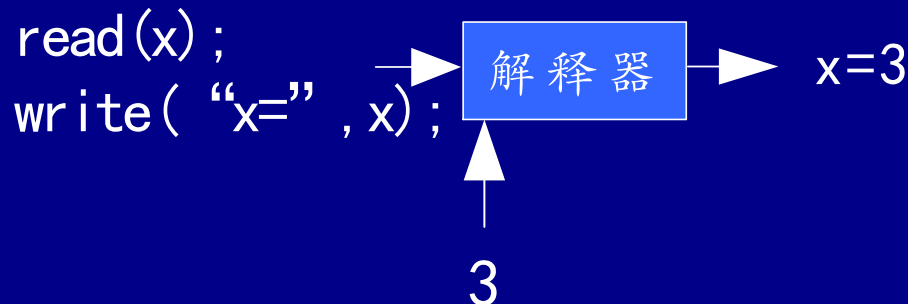
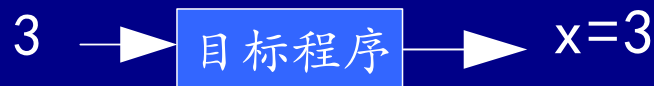
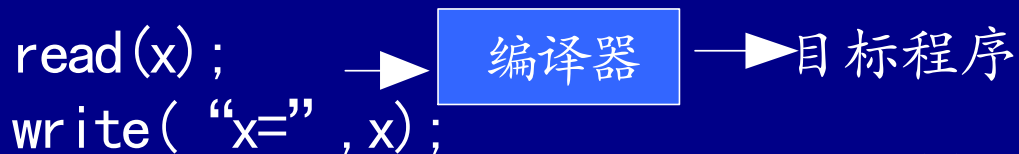
1. 先翻译后执行

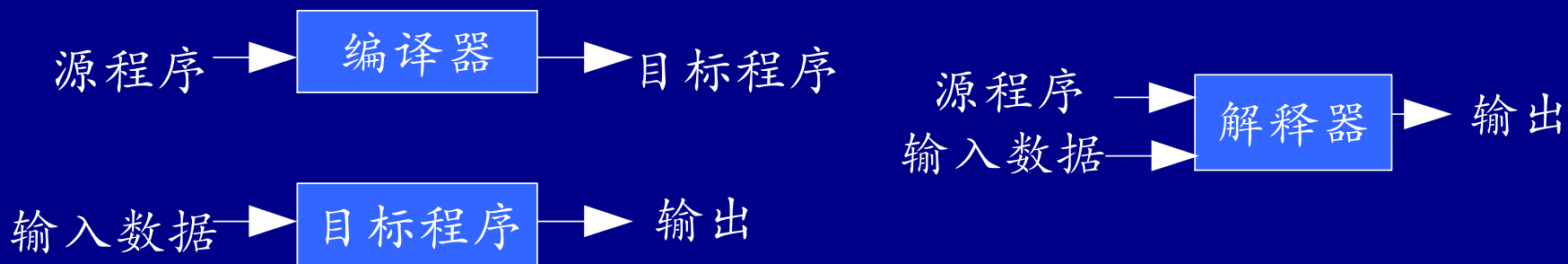


2. 边翻译边执行



例5 假设有源程序: `read(x); write("x=", x);`





解释执行类似于“同声传译”

1. 特点:

- 编译器: 工作效率高, 即时间快、空间省; 交互性与动态特性差、可移植性差。大多数PL采用此种方法翻译;
- 解释器: 工作效率低, 即时间慢、空间费; 交互性与动态特性好、可移植性好。早期的Basic和现在的Python等。

2. 基本功能: 二者相同;

3. 所采用的技术: 从翻译的角度来讲, 两种方式所涉及的原理、方法、技术相似。



1.4 编译器的工作原理与基本组成

1.4.1 通用程序设计语言的主要成份

1. 从语言抽象程度的演变看：

基本结构：过程→模块（ADT，程序包）→类

2. 共同特点：两大部分组成：声明+操作=完整定义

3. 以过程式语言为例：

声明性语句：提供操作对象的性质，如数据类型、值、作用域等；

操作性语句：确定操作的计算次序，完成实际操作。

过程头+过程体=过程定义 例子

4. 编译器对两类语句的翻译：

声明：生成相应的环境，一般是配置存储空间。

操作：生成可执行的代码序列。



1.4.2 以阶段划分编译器

1. 自然语言的翻译过程:

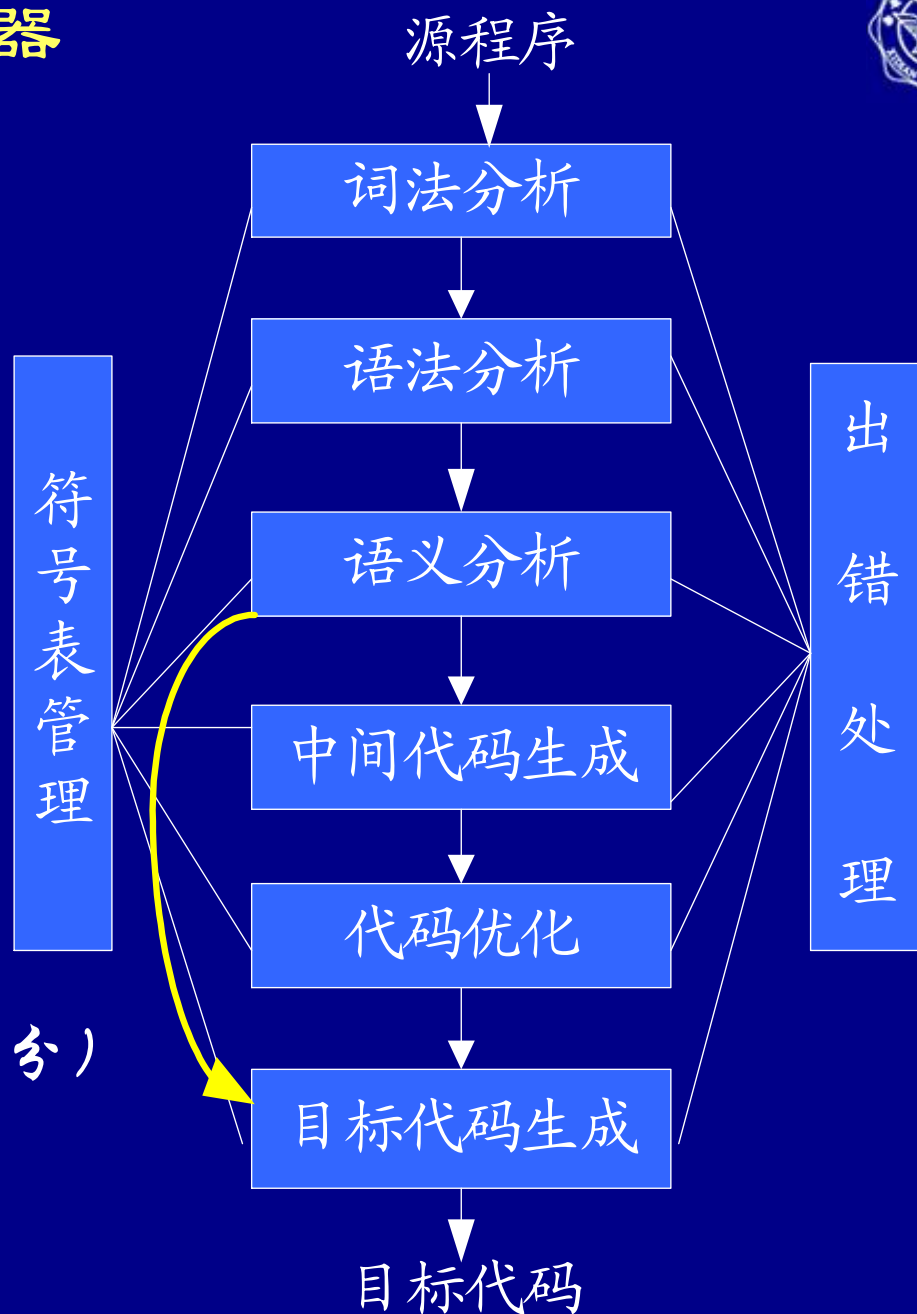
识别单词
识别句子结构
理解意思
译成中文并修饰

2. 编译器的工作过程:

词法分析
语法分析
语义分析
目标代码生成

3. 编译器的阶段 (逻辑模块划分)

4. 中间代码的重要作用





1.4.3 编译器各阶段的工作

例7 Pascal源程序语句如下:

```
var x, y, z : real;
```

```
    x := y + z * 60;
```

(源程序) var x, y, z : real; x := y + z * 60;



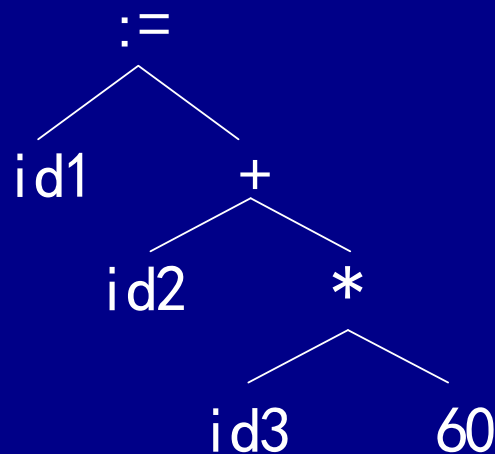
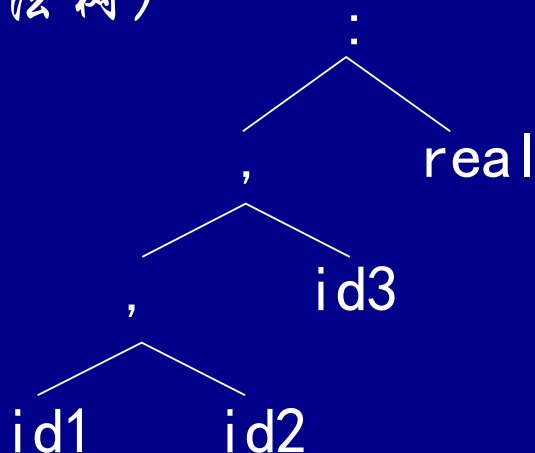
词法分析

(记号流) var id1, id2, id3 : real; id1 := id2+id3*60;



语法分析

(语法树)





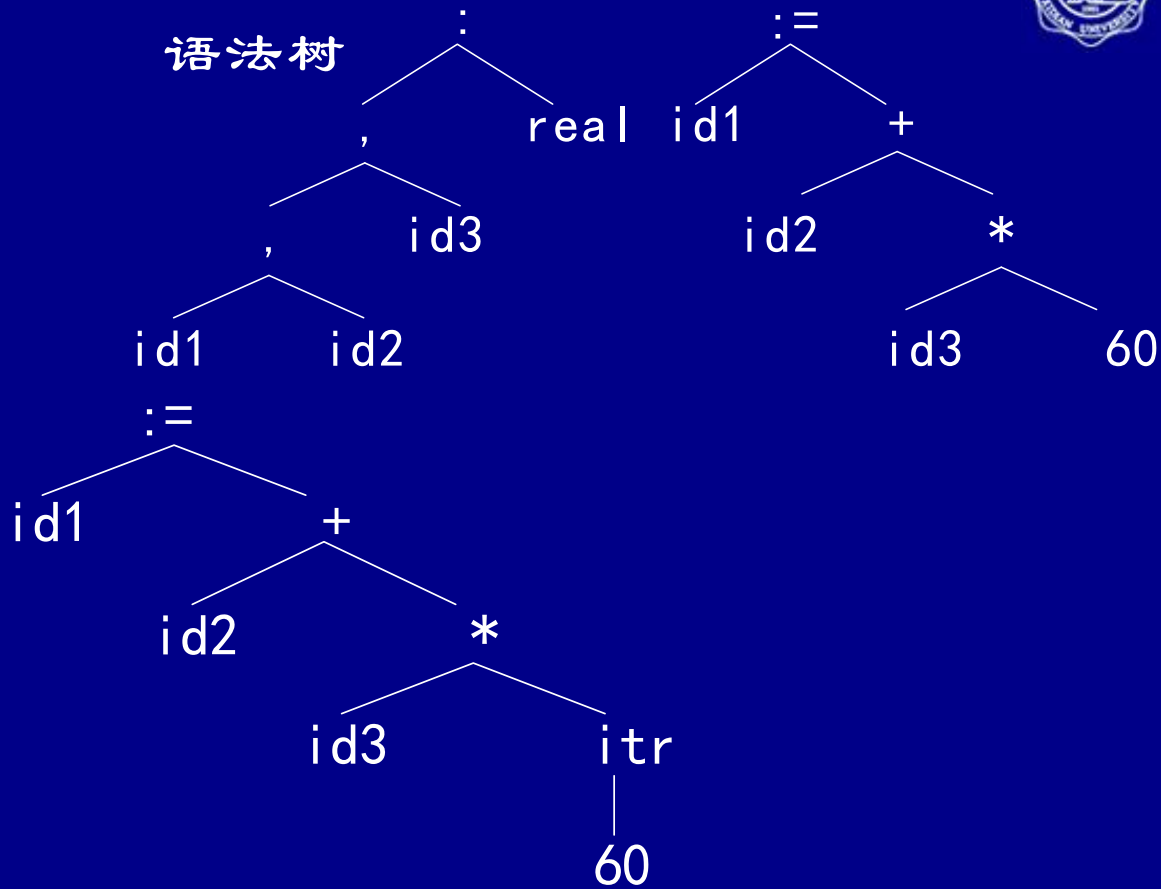
↓ 语义分析

符号表部分内容

1	x	real	0
2	y	real	4
3	z	real	8
4			
		...	...

↓ 中间代码生成

- (1) (itr, 60, , T1)
- (2) (*, id3, T1, T2)
- (3) (+, id2, T2, T3)
- (4) (:=, T3, , id1)



中间代码的形式与作用：

(op, arg1, arg2, result)
result := arg1 op arg2



```
(1) (itr, 60, , T1)
(2) (*, id3, T1, T2)
(3) (+, id2, T2, T3)
(4) (:=, T3, , id1)
```



中间代码优化

```
(1) (*, id3, 60.0, T1)
(2) (+, id2, T1, id1)
```



目标代码生成

```
MOVF    id3,    R2
MULF    #60.0,  R2
MOVF    id2,    R1
ADDF    R2,     R1
MOVF    R1,     id1
```

```
R2 := id3
R2 := R2 * 60.0
R1 := id2
R1 := R1 + R2
id1 := R1
id1 := id2 + id3 * 60.0
x := y + z * 60;
```

1	x	real	0
2	y	real	4
3	z	real	8
4			
		...	...



各阶段工作的归纳

1. **词法分析**: 识别单词, 至少分以下几大类: 关键字 (保留字)、标识符、字面量、特殊符号;
2. **语法分析**: 得到语言结构并以树的形式表示;
3. **语义分析**: 考察结构正确的句子是否语义合法, 修改树结构;
4. **中间代码生成 (可选)**: 生成一种既接近目标语言, 又与具体机器无关的表示, 便于优化与代码生成;
 - (到目前为止, 编译器与解释器可以一致)
5. **中间代码优化 (可选)**: 局部优化、循环优化、全局优化等; 优化实际上是一个等价变换, 变换前后的指令序列完成同样的功能, 但在占用的空间上和程序执行的时间上都更省、更有效。
6. **目标代码生成**: 不同形式—汇编、可重定位、内存形式;
7. **符号表管理**: 记录符号信息并合理组织, 便于各阶段查找、操作等;
8. **出错处理**: 错误的种类—词法错、语法错、静态语义错、动态语义错。



1.4.4 编译器的分析/综合模式

编译器

分析 (前端)

综合 (后端)

源程序

中间代码

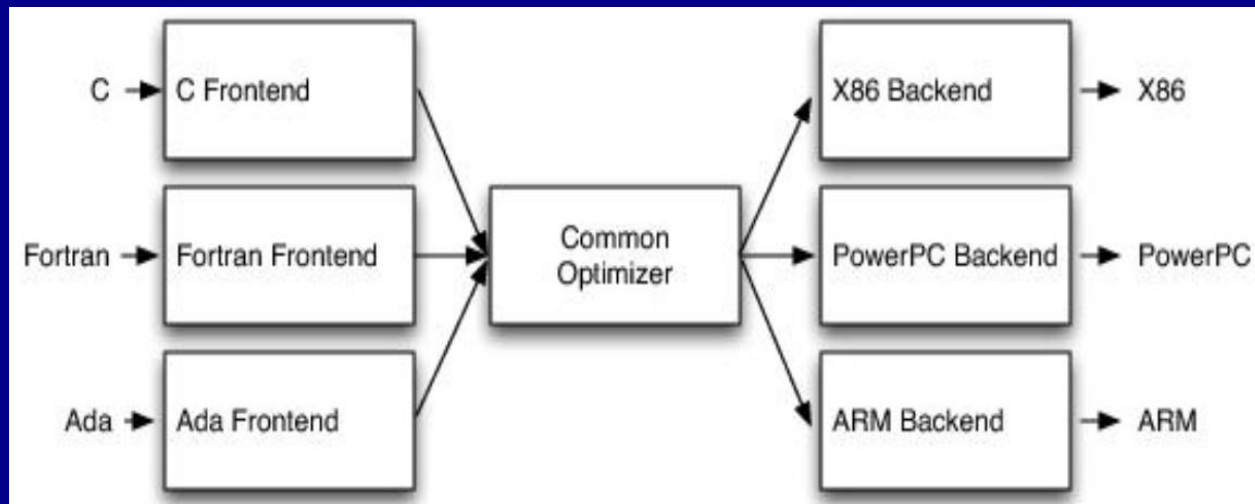
目标代码



1. 前端：语言结构和意义的分析；
2. 后端：语言意义处理；
3. 中间代码：前端与后端的分界；

4. 编译器基础架构 (Infrastructure)

：源代码的中间表示
、一组前端、一组后端；
灵活搭配，提高编译系统的开发效率。





1.4.5 编译器扫描的遍数

1. 将程序完整分析一遍的工作模式称为一遍扫描。

2. 原理上希望扫描遍数越少越好：

- 为编译器的运行提供足够大的内存空间。
- 语言的设计要求，如规定标识符的先声明后引用 源程序

...

```
x := f(a, b, c);
```

...

```
function f(a:integer;  
          b:integer):integer;
```

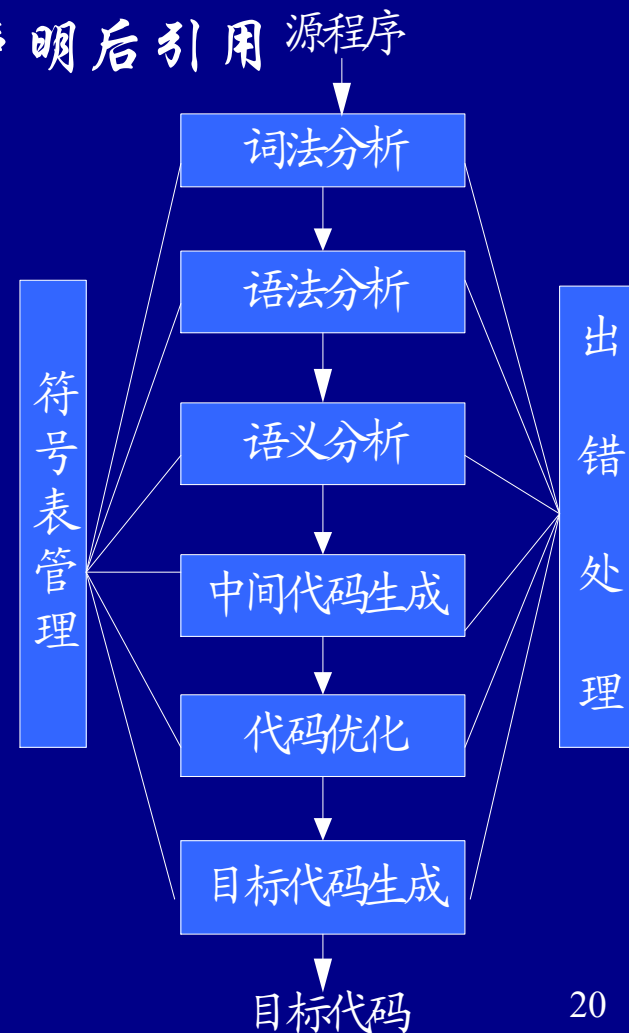
- 编译技术，如拉链-回填

...

```
goto lab1;
```

...

```
lab1: ...
```





1.5 编译器的编写

1. 直接使用汇编语言和程序设计语言;
2. 利用编译器编写工具: 词/语法、语法制导翻译、代码生成、数据流分析等;
3. 基于编译器基础架构的编译器构造系统 (开放式编译器, 如 GCC、SUIF、LLVM等)。

GCC Home Page. <http://gcc.gnu.org>.

Gasta Homepage. <http://gasta.sourceforge.net>

SUIF Compiler System Home Page. <http://suif.stanford.edu>



1.6 本章小结

1. 语言与语言的翻译
2. 编译器与解释器
3. 编译器的基本组成 (工作)
4. 编译器的分析/综合模式, 编译器基础架构
5. 扫描遍数
6. 编译器的编写



第一章 结束



例6 有一Pascal的过程如下所示

```
(1)  procedure sample(y: integer); {过程头}
(2)      var x : integer;          {过程体 (开始) }
(3)      begin x := y;
(4)          if x>100 then x := 0
(5)      end;                      {过程体 (结束) }
```

(1)是过程头，它是一个声明性的语句，为使用者提供调用信息，包括过程名、参数、返回值（如果有的话）等。（接口）

(2)至(5)是过程体，它是一个语句序列。语句序列中既包括声明性语句也包括操作性语句。（实现）

(2)是声明性语句，而(3)至(5)是操作性语句。

对于编译器来讲，它对声明性语句的处理一般是生成相应的环境（存储空间），而对操作性语句则是生成此环境中的可执行代码序列。

为了便于编译器的处理，操作性语句中使用的每个操作对象，均应在使用前声明，即符合先声明后引用的原则。